

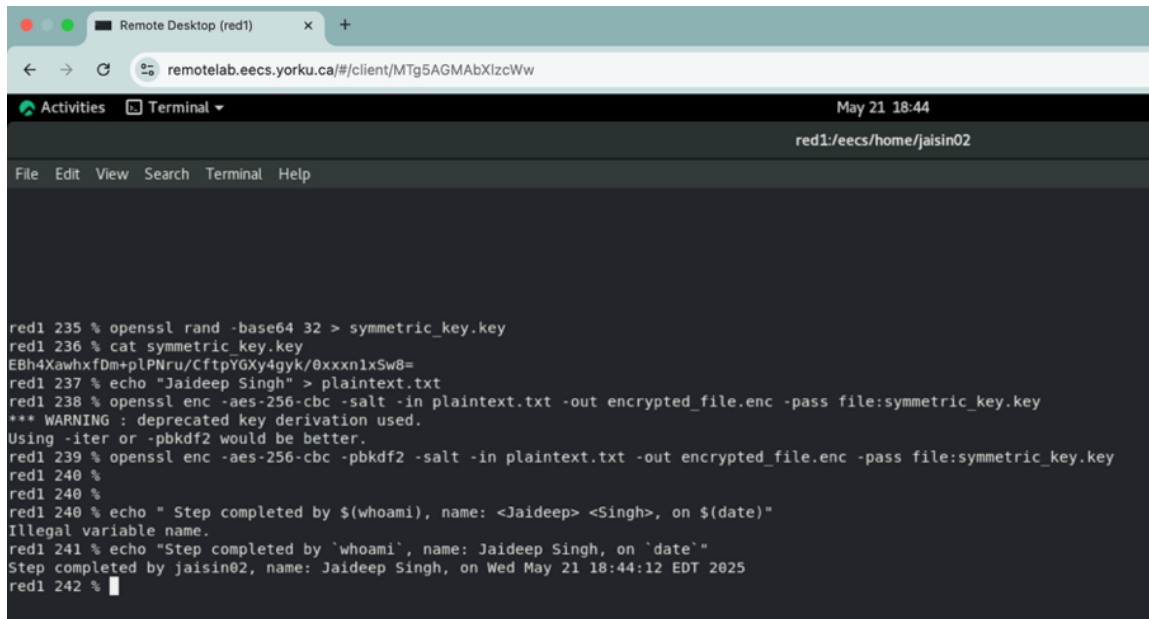
EECS 3482 – Introduction to Computer Security

Lab 1- Symmetric and Asymmetric Cryptography

Submitted by: Jaideep Singh | Student ID:218472241

Part 1

1. Key Generation and Encryption



The screenshot shows a Remote Desktop window titled 'Remote Desktop (red1)'. The address bar displays 'remotelab.eecs.yorku.ca/#/client/MTg5AGMAbXlzcWw'. The terminal window shows the following commands and output:

```
red1 235 % openssl rand -base64 32 > symmetric_key.key
red1 236 % cat symmetric_key.key
EBh4XawhxfDm+plPNru/CftpYGXy4gyk/0xxn1xSw8=
red1 237 % echo "Jaideep Singh" > plaintext.txt
red1 238 % openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted_file.enc -pass file:symmetric_key.key
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
red1 239 % openssl enc -aes-256-cbc -pbkdf2 -salt -in plaintext.txt -out encrypted_file.enc -pass file:symmetric_key.key
red1 240 %
red1 240 %
red1 240 % echo "Step completed by $(whoami), name: <Jaideep> <Singh>, on $(date)"
Illegal variable name.
red1 241 % echo "Step completed by `whoami`, name: Jaideep Singh, on `date`"
Step completed by jaisin02, name: Jaideep Singh, on Wed May 21 18:44:12 EDT 2025
red1 242 %
```

Question 1: What is the role of the -salt flag in AES encryption?

Ans: AES encryption's `-salt` flag strengthens security by incorporating random data throughout the key derivation process. The `-salt` option adds a unique salt value during file encryption using a password or key file. Further, the application of salt stops identical plaintext and password combinations from producing matching ciphertexts which helps protect against dictionary attacks and precomputed rainbow table attacks. Repeated encryptions moreover using the same key without a salt would produce identical outputs which surely expose patterns to attackers. The use of the `-salt` flag guarantees that different encryption results occur for each encryption operation despite repeating the same key.

2. Decryption

Question 2: What happens if you attempt to decrypt the file with the wrong key?

Answer2: Decrypting a file with an incorrect key will finish successfully without any errors but the decrypted file's contents will appear as unreadable garbage. AES decryption further performs operations based on the given key without checking if it is correct. The symmetric nature of AES means that an incorrect key which produces improper encrypted data transformation which results in meaningless output.

```
Remote Desktop (red1) x +
remotelab.eecs.yorku.ca/#/client/MTg5AGMabXlzcWw
May 21 18:50
red1:/eecs/home/jaisin02

File Edit View Search Terminal Help

red1 235 % openssl rand -base64 32 > symmetric_key.key
red1 236 % cat symmetric_key.key
EBh4XawhxfDm+plPNru/CftpYGYy4gyk/0xxxnlxSw8=
red1 237 % echo "Jaideep Singh" > plaintext.txt
red1 238 % openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted_file.enc -pass file:symmetric_key.key
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
red1 239 % openssl enc -aes-256-cbc -pbkdf2 -salt -in plaintext.txt -out encrypted_file.enc -pass file:symmetric_key.key
red1 240 %
red1 240 %
red1 240 % echo " Step completed by $(whoami), name: <Jaideep> <Singh>, on $(date)"
Illegal variable name.
red1 241 % echo "Step completed by `whoami`, name: Jaideep Singh, on `date`"
Step completed by jaisin02, name: Jaideep Singh, on Wed May 21 18:44:12 EDT 2025
red1 242 % openssl enc -d -aes-256-cbc -pbkdf2 -in encrypted_file.enc -out decrypted_file.txt -pass file:symmetric_key.key
red1 243 % cat decrypted_file.txt
Jaideep Singh
red1 244 % diff plaintext.txt decrypted_file.txt
red1 245 % echo "Step completed by `whoami`, name: Jaideep Singh, on `date`"
Step completed by jaisin02, name: Jaideep Singh, on Wed May 21 18:49:36 EDT 2025
red1 246 %
```

3. Performance and Key Size Experiment

```
Remote Desktop (red1) x +
remotelab.eecs.yorku.ca/#/client/MTg5AGMabXlzcWw
May 21 19:03
red1:/eecs/home/jaisin02

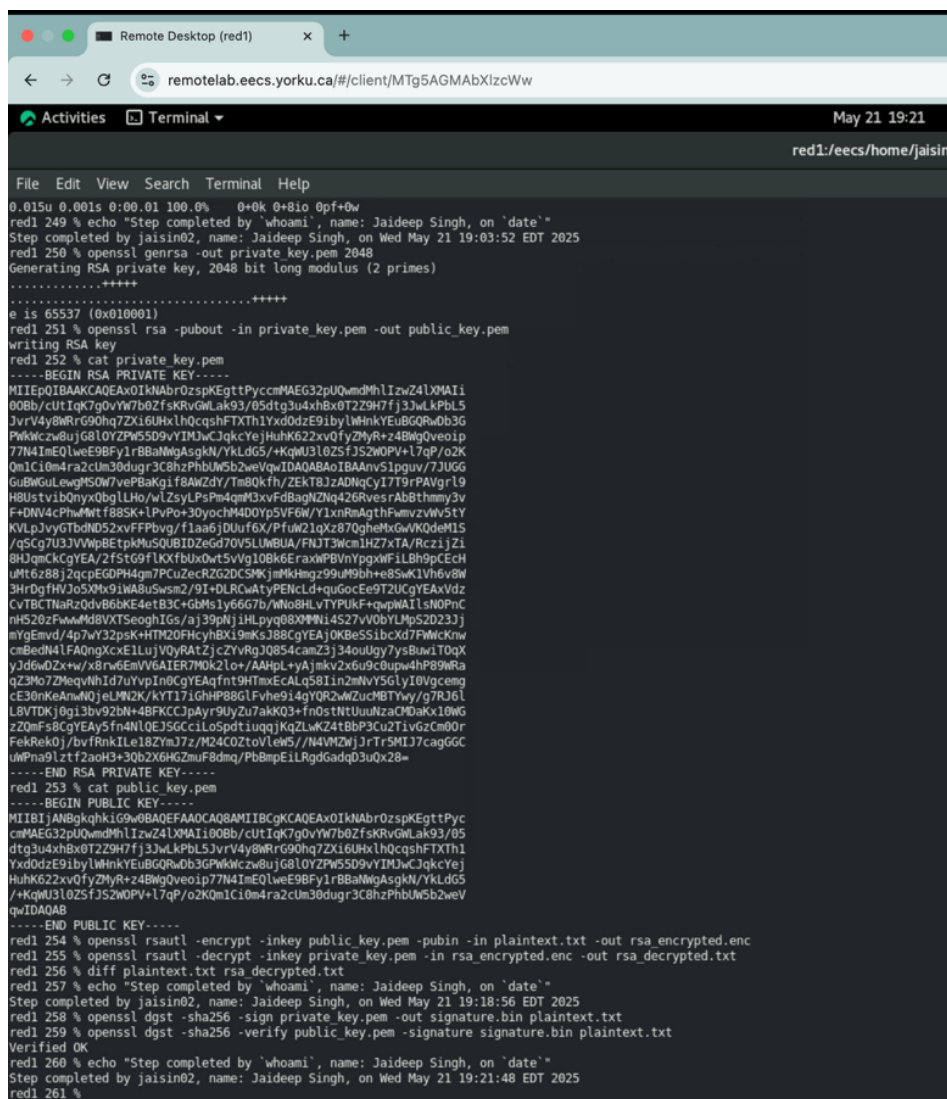
File Edit View Search Terminal Help

red1 235 % openssl rand -base64 32 > symmetric_key.key
red1 236 % cat symmetric key.key
EBh4XawhxfDm+plPNru/CftpYGYy4gyk/0xxxnlxSw8=
red1 237 % echo "Jaideep Singh" > plaintext.txt
red1 238 % openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted_file.enc -pass file:symmetric_key.key
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
red1 239 % openssl enc -aes-256-cbc -pbkdf2 -salt -in plaintext.txt -out encrypted_file.enc -pass file:symmetric_key.key
red1 240 %
red1 240 %
red1 240 % echo " Step completed by $(whoami), name: <Jaideep> <Singh>, on $(date)"
Illegal variable name.
red1 241 % echo "Step completed by `whoami`, name: Jaideep Singh, on `date`"
Step completed by jaisin02, name: Jaideep Singh, on Wed May 21 18:44:12 EDT 2025
red1 242 % openssl enc -d -aes-256-cbc -pbkdf2 -in encrypted_file.enc -out decrypted_file.txt -pass file:symmetric_key.key
red1 243 % cat decrypted_file.txt
Jaideep Singh
red1 244 % diff plaintext.txt decrypted_file.txt
red1 245 % echo "Step completed by `whoami`, name: Jaideep Singh, on `date`"
Step completed by jaisin02, name: Jaideep Singh, on Wed May 21 18:49:36 EDT 2025
red1 246 % time openssl enc -aes-128-cbc -pbkdf2 -in plaintext.txt -out enc128.enc -pass file:symmetric_key.key
0.007u 0.002s 0:00.01 0.0% 0+0k 0+8io 0pf+0w
red1 247 % time openssl enc -aes-192-cbc -pbkdf2 -in plaintext.txt -out enc192.enc -pass file:symmetric_key.key
0.014u 0.001s 0:00.01 100.0% 0+0k 0+8io 0pf+0w
red1 248 % time openssl enc -aes-256-cbc -pbkdf2 -in plaintext.txt -out enc256.enc -pass file:symmetric_key.key
0.015u 0.001s 0:00.01 100.0% 0+0k 0+8io 0pf+0w
red1 249 % echo "Step completed by `whoami`, name: Jaideep Singh, on `date`"
Step completed by jaisin02, name: Jaideep Singh, on Wed May 21 19:03:52 EDT 2025
red1 250 %
```

Question 3: How does the encryption time vary with key size? What trade-offs do you observe? Is the time command precise enough to show you the difference between the three commands?

Answer: When AES key size increases from 128-bit to 192-bit and subsequently to 256-bit, encryption time increases. Larger keys need more transformation rounds: AES-128 utilizes 10, AES-192 and AES-256 12 and 14. The time difference between encryption processes is minor for plaintext data but significant for big files or time-sensitive applications. AES-128 balances security and performance by encrypting quickly for many applications. AES-192 and AES-256 provide high-security encryption at a somewhat slower pace. The time command frequently displays performance differences in big or complicated files with sufficient accuracy. Time differences are too tiny to discern on modern technology when testing small files like plaintext.txt with merely a name. Hyperfine benchmarking tools or repeated runs and averaging results may improve short task measurement.

Part 2



```

Remote Desktop (red1) x +
remotelab.eecs.yorku.ca/#/client/MTg5AGMabXlzcWw
May 21 19:21
red1:/eecs/home/jaisin0

File Edit View Search Terminal Help
0.015u 0.001s 0:00.01 100.0% 0+0k 0+810 0pf+0w
red1 249 % echo "Step completed by 'whoami', name: Jaideep Singh, on 'date'"
Step completed by jaisin02, name: Jaideep Singh, on Wed May 21 19:03:52 EDT 2025
red1 250 % openssl genrsa -out private_key.pem 2048
Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.....+++++
e is 65537 (0x010001)
red1 251 % openssl rsa -pubout -in private_key.pem -out public_key.pem
writing RSA key
red1 252 % cat private_key.pem
-----BEGIN RSA PRIVATE KEY-----
MIIEpQIBAAQCAQEAx0IkNAbr0zspKEgttPyccmMAEG32pUQwmdMhLIzwZ4LXMAIi
00Bb/cutIqK7g0vYw7b0ZfsKRVgWLaK93/05dtg3u4xhBx0T29H7fj3JwLkPbL5
Jvrv4y8MRrG90hq7ZX16UhxLh0cqsHFTXTh1Yx0ddzE91byLWnkYeuBGQRw0b3G
PwKwczw8ujG810Y2Pw5S09vYIMJwCJqkCyejHuhK622xv0fyZMyR+z4BwQv0veoip
77N4ImEQlwe9BFy1rBBAWpAsqkN/YkLdG5/+KqWU310ZsfJ52W0PV+l7qP/o2K
Qm1C10m4ra2cUm30dugr3C8hzPhbUw5b2weVqWIDAQAABoIBAAmV51pguv/7JUGG
6uBwGullwe9BS0W7veBpKqif8AWZ0Y/Tm0kfh/2EATBjzADlqCy1779rPAWgrl9
H8UsTvlb0nyx0BgLLHo/vLZsyLPsPm4qmM3xvFdBqgNZnq426RvesrAbBthamy3v
F+DNV4cPhwMt1f8BSK+LP+Po+30yocM4D0Yp5VF6W/Y1xnRaAgthFwmvzWv5TY
KvLpJvyGtBdN52xvFFPbvq/f1aa6jD0uf6X/PfWu21qx2870gheHxGwK0deM15
/qSG7U3JVVWpBETpkMuS0UBIDZegd70V5LUMBUA/FNJT3wcm1H27xTA/RcziJZl
8HJqmCkCYEA/2fstG9fLkXfbUx0vt5vVg10Bk6ErxwPBVnYpgxwF1L8h9pCEch
uMt6z8Bj2qccEGDPH4gm7PCuZecRZG2DCSMKjaMKHmgz99uM9bh+e8SwK1Vh6v8W
3HrdGfhVJo5XmX9IWA8uSwsm2/9I+DLRCwAtyPENcl+quGocEe9T2UCgYEAxVdz
CVTBCtNaRzQdvB6bKE4etB3C+GbMs1y66G7b/wNo8HLvTYPukF+qwpwAILsN0PnC
nh520zFwwMdB8VXTSeoghIGs/aj39pNjiHLpyq08X0PwN14527vV0bYLMp52023Jj
eYgEmvd/4p7wY32psK+HTM20FHcyhBX19mKsJ88CYEAj0KBe5SibcX07FwMcKw
cmBedN4lFAQngXcxELUjVQyRATZjcZYVRgJ0854camZ3j34ouUgy7ysBuwlTQqX
yJ3d6w0Zxrw/x8rw6EmVVA6IER7M0k2Lo+/AAHPL+YAjmKv2x6u9c0upw4hP89WRa
qZ3Mo7ZMeqVhId7uYvpIn0CgYEAqfnt9HTmxECALq58IIn2mNvY5GlyI0Vgcmg
cE30nKeAnwN0jeLMN2K/KYt17iGHP88GLFvhe91qYQR2wWZucMBTYw/g7Rj6L
L8VTDKj0gi3bv92bN+4BFKCCJpAyr9UyZu7akkQ3+fn0stNtUuuNzaCMAKx10WG
z2QmF8CgYEAy5fn4NlQEJSGCCiLoSptdiuqqjKqZLwK24tBbP3Cu2T1vGzCm00r
FekRek0j/bvfrnkILe18ZymJ7z/M24C0ZtoVLeW5//N4VMZWjJrTrSMIJ7cagGGC
uMPna9Lztzf2aoH3+3QbZX6H6ZmuF8dmq/PbBmpE1LRgdGadQ03Uqx28=
-----END RSA PRIVATE KEY-----
red1 253 % cat public_key.pem
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAx0IkNAbr0zspKEgttPyccmMAEG32pUQwmdMhLIzwZ4LXMAIi00Bb/cutIqK7g0vYw7b0ZfsKRVgWLaK93/05dtg3u4xhBx0T29H7fj3JwLkPbL5Jvrv4y8MRrG90hq7ZX16UhxLh0cqsHFTXTh1Yx0ddzE91byLWnkYeuBGQRw0b3GPwKwczw8ujG810Y2Pw5S09vYIMJwCJqkCyejHuhK622xv0fyZMyR+z4BwQv0veoip77N4ImEQlwe9BFy1rBBAWpAsqkN/YkLdG5/+KqWU310ZsfJ52W0PV+l7qP/o2KQm1C10m4ra2cUm30dugr3C8hzPhbUw5b2weVqWIDAQAAB
-----END PUBLIC KEY-----
red1 254 % openssl rsautl -encrypt -inkey public_key.pem -pubin -in plaintext.txt -out rsa_encrypted.enc
red1 255 % openssl rsautl -decrypt -inkey private_key.pem -in rsa_encrypted.enc -out rsa_decrypted.txt
red1 256 % diff plaintext.txt rsa_decrypted.txt
red1 257 % echo "Step completed by 'whoami', name: Jaideep Singh, on 'date'"
Step completed by jaisin02, name: Jaideep Singh, on Wed May 21 19:18:56 EDT 2025
red1 258 % openssl dgst -sha256 -sign private_key.pem -out signature.bin plaintext.txt
red1 259 % openssl dgst -sha256 -verify public_key.pem -signature signature.bin plaintext.txt
Verified OK
red1 260 % echo "Step completed by 'whoami', name: Jaideep Singh, on 'date'"
Step completed by jaisin02, name: Jaideep Singh, on Wed May 21 19:21:48 EDT 2025
red1 261 %

```

Question 4: Why is RSA encryption slower compared to AES?

Answer: Due to sophisticated mathematical computations, RSA is slower than AES due to operational differences. The RSA algorithm combines complex mathematical calculations with big prime numbers and modular exponentiation to encrypt asymmetrically. These

procedures take longer with bigger key sizes like 2048-bit or 4096-bit. AES uses fast and lightweight bitwise XORs, replacements, and permutations in fixed-size data blocks for symmetric encryption. AES encryption is ideal for real-time applications and bulk data encryption since it processes massive data sets quickly. In a hybrid encryption configuration, RSA protects AES keys while AES encrypts data due to their performance differences. The method maximizes algorithm capabilities by combining RSA's security and AES's efficiency.

Question 5: What role does hashing play in digital signatures?

Answer: Hashing protects data and speeds up processing in digital signatures. SHA-256 produces a unique fixed-size digest of the file contents instead than signing it. The digital signature is created by encrypting the hash using the sender's private key. The system signs the hash instead of the complete file, making hashing faster and more effective. Tampering is obvious since even the tiniest file content change produces a new hash. Data integrity may be maintained by hashing the received file and comparing it to the decrypted signature. Hashing improves performance, secures digital signatures, and detects even slight data modifications.

Part 3

Files attached for part 3.